

Coarsen-Solve-Reconstruct

- Processing time만 coarsen: $p'_{ij} := \left\lceil \frac{p_{ij}}{\tau} \right\rceil$
 - Due window는 원본 유지
- Procedure
 - Initialize
 - [순방향, 역방향] x [wxd2, WSPT, wxd7] → 6개 schedule에 후처리 후 best schedule 선택
 - 순방향: Idle time insertion
 - 역방향: Left-justify + idle time insertion
 - Neighborhood search
 - CP with warm-start
 - 후처리: Left-justify + idle time insertion

Coarsen-Solve-Reconstruct

Idle Time Insertion

◦ 마지막 stage의 machine 별로(m) 반복:

▫ $j := n_m$

▫ While $j > 0$:

① 해당 operation을 최좌측 operation으로 하는 block(연속한 operation의 set, S_M) 생성

② Δ_2 정의:

- Block의 오른쪽에 job이 있는 경우 해당 간격
- 아닌 경우 ∞

③ S_M 을 partition

- $S_E = \{j | C_j < d_j^-, j \in S_M\}$ (definitely early)
- $S_D = \{j | d_j^- \leq C_j < d_j^+, j \in S_M\}$ (on-time, right-room)
- $S_T = \{j | d_j^+ \leq C_j, j \in S_M\}$ (tardy if right-shifted)

④ $\sum_{j \in S_E} w_j^- > \sum_{j \in S_T} w_j^+$
(If right-shifting reduces obj. func. contribution)

▪ True: right-shift

- $\Delta_1 := \min \left\{ \min_{j \in S_E} (d_j^- - C_j), \min_{j \in S_D} \{d_j^+ - C_j\} \right\}$
- Insert $\Delta := \min\{\Delta_1, \Delta_2\}$ time units before S_M
($C_k := C_k + \Delta \forall k \in S_M$)

▪ False: go to the next(left) job

- $j := j - 1$

Coarsen-Solve-Reconstruct

Idle Time Insertion After Coarsening – Multiplication

◦ 마지막 stage의 machine 별로(m) 반복:

▫ $j := n_m$

▫ While $j > 0$:

① 해당 operation을 최좌측 operation으로 하는 block (연속한 operation의 set, S_M) 생성

② Δ_2 정의:

- Block의 오른쪽에 job이 있는 경우 해당 간격
- 아닌 경우 ∞

③ S_M 을 partition

- $S_E = \{j | K \cdot C_j < d_j^-, j \in S_M\}$ (definitely early)
- $S_D = \{j | d_j^- \leq K \cdot C_j < d_j^+, j \in S_M\}$ (on-time, right-room)
- $S_T = \{j | d_j^+ \leq K \cdot C_j, j \in S_M\}$ (tardy if right-shifted)

④ $\sum_{j \in S_E} w_j^- > \sum_{j \in S_T} w_j^+$
(If right-shifting reduces obj. func. contribution)

▪ True: right-shift

$$\Delta_1 := \min \left\{ \min_{j \in S_E} (d_j^- - K \cdot C_j), \min_{j \in S_D} (d_j^+ - K \cdot C_j) \right\}$$

- Insert $\Delta := \min\{\Delta_1, \Delta_2\}$ time units before S_M
($C_k := C_k + \Delta \forall k \in S_M$)

▪ False: go to the next(left) job

- $j := j - 1$

Coarsen-Solve-Reconstruct

Idle Time Insertion After Coarsening – Division

◦ 마지막 stage의 machine 별로(m) 반복:

▫ $j := n_m$

▫ While $j > 0$:

① 해당 operation을 최좌측 operation으로 하는 block (연속한 operation의 set, S_M) 생성

② Δ_2 정의:

- Block의 오른쪽에 job이 있는 경우 해당 간격
- 아닌 경우 ∞

③ S_M 을 partition

- $S_E = \{j | C_j < (d_j^- \div K), j \in S_M\}$ (definitely early)
- $S_D = \{j | (d_j^- \div K) \leq C_j < (d_j^+ \div K), j \in S_M\}$ (on-time, right-room)
- $S_T = \{j | (d_j^+ \div K) \leq C_j, j \in S_M\}$ (tardy if right-shifted)

④ $\sum_{j \in S_E} w_j^- > \sum_{j \in S_T} w_j^+$
(If right-shifting reduces obj. func. contribution)

▪ True: right-shift

$$\Delta_1 := \min \left\{ \min_{j \in S_E} ((d_j^- \div K) - C_j), \min_{j \in S_D} ((d_j^+ \div K) - C_j) \right\}$$

- Insert $\Delta := \min\{\Delta_1, \Delta_2\}$ time units before S_M
($C_k := C_k + \Delta \forall k \in S_M$)

▪ False: go to the next(left) job

- $j := j - 1$

Coarsen-Solve-Reconstruct

Idle Time Insertion After Coarsening – Ceiling (?)

◦ 마지막 stage의 machine 별로(m) 반복:

▫ $j := n_m$

▫ While $j > 0$:

① 해당 operation을 최좌측 operation으로 하는 block (연속한 operation의 set, S_M) 생성

② Δ_2 정의:

- Block의 오른쪽에 job이 있는 경우 해당 간격
- 아닌 경우 ∞

③ S_M 을 partition

- $S_E = \{j | C_j < \lceil d_j^- \div K \rceil, j \in S_M\}$ (definitely early)
- $S_D = \{j | \lceil d_j^- \div K \rceil \leq C_j < \lceil d_j^+ \div K \rceil, j \in S_M\}$ (on-time, right-room)
- $S_T = \{j | \lceil d_j^+ \div K \rceil \leq C_j, j \in S_M\}$ (tardy if right-shifted)

④ $\sum_{j \in S_E} w_j^- > \sum_{j \in S_T} w_j^+$
(If right-shifting reduces obj. func. contribution)

▪ True: right-shift

$$\Delta_1 := \min \left\{ \min_{j \in S_E} (\lceil d_j^- \div K \rceil - C_j), \min_{j \in S_D} (\lceil d_j^+ \div K \rceil - C_j) \right\}$$

- Insert $\Delta := \min\{\Delta_1, \Delta_2\}$ time units before S_M
($C_k := C_k + \Delta \forall k \in S_M$)

▪ False: go to the next(left) job

- $j := j - 1$

Coarsen-Solve-Reconstruct

Idle Time Insertion After Coarsening – Floor for Shifting

◦ 마지막 stage의 machine 별로(m) 반복:

▫ $j := n_m$

▫ While $j > 0$:

① 해당 operation을 최좌측 operation으로 하는 block (연속한 operation의 set, S_M) 생성

② Δ_2 정의:

- Block의 오른쪽에 job이 있는 경우 해당 간격
- 아닌 경우 ∞

③ S_M 을 partition

- $S_E = \{j | \lfloor K \rfloor C_j < d_j^-, j \in S_M\}$ (definitely early)
- $S_D = \{j | d_j^- \leq \lfloor K \rfloor C_j < d_j^+, j \in S_M\}$ (on-time, right-room)
- $S_T = \{j | d_j^+ \leq \lfloor K \rfloor C_j, j \in S_M\}$ (tardy if right-shifted)

④ $\sum_{j \in S_E} w_j^- > \sum_{j \in S_T} w_j^+$
(If right-shifting reduces obj. func. contribution)

▪ True: right-shift

$$\Delta_1 := \min \left\{ \min_{j \in S_E} (\lfloor d_j^- \div K \rfloor - C_j), \min_{j \in S_D} (\lfloor d_j^+ \div K \rfloor - C_j) \right\}$$

- Insert $\Delta := \min\{\Delta_1, \Delta_2\}$ time units before S_M
($C_k := C_k + \Delta \forall k \in S_M$)

▪ False: go to the next(left) job

- $j := j - 1$

Flooring may be inefficient
but is safe against overshooting